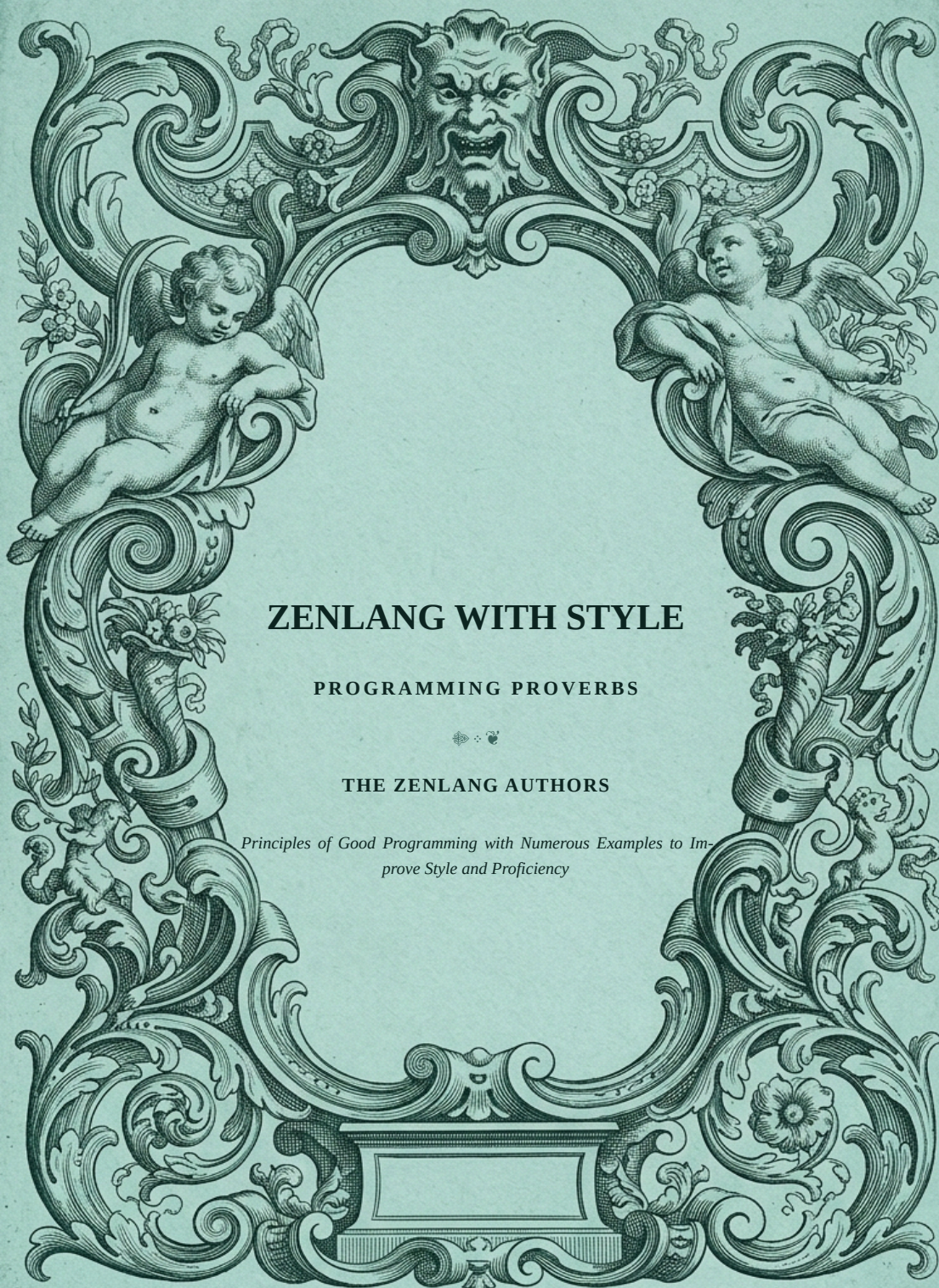




ZENLANG WITH STYLE

PROGRAMMING PROVERBS



THE ZENLANG AUTHORS

Principles of Good Programming with Numerous Examples to Improve Style and Proficiency

[HAYDEN / ZEN PRESS • FIRST EDITION]

Foreword: The Zen of Code



Most programming languages force you into a compromise:

- ❖ **High-level scripting languages** (Python, Ruby, JavaScript) offer rapid prototyping and ergonomic data manipulation, but struggle with terminal latency, distribution overhead, and single-binary packaging.
- ❖ **Systems languages** (C, C++, Rust, Zig) deliver blisteringly fast, sub-millisecond execution and tiny native binaries, but require extensive boilerplate for daily scripts and UI layout.

Zenlang bridges this divide.

It combines visual data flow (`->` and `<-`), first-class terminal canvas rendering, dual execution parity (interpret instantly during development or compile directly to native C with `-g`), and explicit memory management.

❖ C H A P T E R ❖

Table of Contents



Foreword: The Zen of Code	2
Getting Started	6
Installation	6
Your First Program: “Hello, Zen!”	6
The Interactive Playground	7
The Terminal File Manager (zenfm)	7
Formatting Code (zenfmt)	7
Visual Data Flow	7
The Assignment Arrow (->)	7
The Return Arrow (<-)	8
Rebinding & Mutation	8
Types, Collections & Sentinels	9
Primitive Types	9
Sentinels: Nothing and Default	9
Lists	10
Maps	10
Control Flow & Looping	11
Branching with when	11
Looping with do while and do until	12
Pattern Matching & Destructuring	12
Functions & Closures	14
Defining Functions	14
First-Class & Anonymous Functions	14
Closures & Snapshot-By-Value Capture	15
Structures, Classes & OOP	15
Structures (struct)	15
Classes & Object-Oriented Programming (class)	16
Algebraic Data Types (ADTs)	17
Error Handling & Recovery	18
Catching Errors with check	18
Raising Errors with raise	18
The Result Map Pattern	18
Modules, Packages & Reflection	19
Modules & Packages (use)	19
The Built-in module Object	19
Runtime Reflection with zen.reflect	20
Terminal Mastery & UI	22
Terminal Control (zen.sys.term)	22
Declarative Tables (zen.ui.table)	22
Animated Spinners (zen.ui.spinner)	23

Visual Charts & Sparklines (<code>zen.ui.chart</code>)	23
Dual Execution & The C Runtime	24
The Two Execution Modes	24
Under the Hood: C Transpilation & SMIR	24
The Shared C Runtime (<code>runtime/</code>)	24
The Road to Self-Hosting (<code>selfhost/</code>)	25
Practical Projects	25
Project 1: Unix Log Analyzer (<code>zlog.zl</code>)	25
Project 2: Interactive Terminal Task Manager (<code>ztodo.zl</code>)	27

PART I

Language Fundamentals

Installation, visual data flow, types, sentinels, and expressive control flow.

❖ C H A P T E R ❖

Getting Started



Welcome to your journey with Zenlang! In this chapter, you will install the language tools, write and run your first Zenlang program, and learn how to use the built-in developer tools.

Installation

Zenlang uses a centralized driver script located at `./scripts/zen`.

To install the `zen` executable into your local `./bin` directory:

```
git clone https://github.com/zenlang/zen.git
cd zen
./scripts/zen install
```

Verify that Zenlang is installed:

```
./bin/zen --help
```

Your First Program: “Hello, Zen!”

Create a new file named `hello.zl`:

```
// hello.zl
use zen.io

function main() {
    io.writeln(`Hello, World from Zenlang!`)
    <- 0
}
```

Running with the Interpreter

During development, run scripts instantly with the interpreter:

```
./bin/zen hello.zl
```

Compiling to Native C (-g)

Zenlang features **dual execution parity**. The exact same file can be compiled directly into a standalone, optimized native executable:

```
./bin/zen -g hello.zl
```

This generates C code, invokes the C compiler (`gcc` or `clang`), links the runtime, and executes the native binary in milliseconds.

The Interactive Playground

Zenlang includes an interactive terminal code lab with real-time syntax highlighting, templates, and live execution:

```
./scripts/zen playground
```

Playground Shortcuts:

- ❖ **Ctrl+R**: Run code immediately
- ❖ **Ctrl+P**: Load standard library template examples
- ❖ **Ctrl+E**: Switch execution engine (Interpreter / Native -g)
- ❖ **Ctrl+S**: Save file
- ❖ **Ctrl+Q**: Quit

The Terminal File Manager (zenfm)

To explore your project files, browse directories, and preview .zl source files with live syntax highlighting:

```
./scripts/zen fm .
```

Press **P** on any .zl file to open it directly in the Playground!

Formatting Code (zenfmt)

Zenlang includes an automated style formatter:

```
./scripts/zen fmt -w hello.zl
```

❖ C H A P T E R ❖

Visual Data Flow



At the core of Zenlang is the philosophy of **Visual Data Flow**. In most languages, assignment uses = and returning uses return. In Zenlang, data movement is explicitly visible through arrows:

- ❖ -> represents **assignment, binding, and data flow forward**.
- ❖ <- represents **returning, yielding, and data flow outward**.

The Assignment Arrow (->)

To declare a new variable or rebind an existing one, use let with ->:

```
use zen.io
use zen.text.string as Str

function main() {
  let name -> `Alice`
  let age -> 30
  let is_member -> True

  io.writeln(`Name: ` + name)
  io.writeln(`Age: ` + Str.to_string(age))
  <- 0
}
```

Why Arrows Instead of =?

1. **Reads Left-to-Right:** `let value -> compute()` explicitly visualizes the output of `compute()` flowing into `value`.
2. **Distinguishes Assignment from Equality:** In Zenlang, `==` tests equality and `->` assigns. There is no confusing `=` assignment with comparison.

The Return Arrow (<-)

To return a value from a function, use `<-`:

```
function add(a, b) {
  <- a + b
}

function multiply(a, b) {
  let product -> a * b
  <- product
}
```

A function can return early at any point:

```
function clamp(val, min_val, max_val) {
  when val < min_val { <- min_val }
  when val > max_val { <- max_val }
  <- val
}
```

Rebinding & Mutation

Variables in Zenlang can be rebound:

```
let score -> 100
score -> score + 50
score -> score * 2
```

Loop Index Rebinding Pattern

When updating loop counters across iterations, compute indices with clean arithmetic expressions:

```
let i -> 0
do while i < 5 {
  io.writeln(`Step: ` + Str.to_string(i))
  i -> 0 + i + 1
}
```

❖ C H A P T E R ❖

Types, Collections & Sentinels



Zenlang features a flexible, expressive type system designed for reliability and ease of use.

Primitive Types

Type	Examples	Description
Integer	42, -10, 0	64-bit signed integers
Decimal	3.14159, 0.0, -0.5	64-bit IEEE floating-point numbers
Boolean	True, False	Truth values (also lowercase true, false)
String	`hello`, `multiline\nstring`	UTF-8 text strings

String Literals

Strings in Zenlang are enclosed in backticks (`...`):

```
use zen.io
use zen.text.string as Str

function main() {
  let msg -> `Hello\nWorld!`
  let len -> Str.length(msg)
  let upper -> Str.to_uppercase(msg)

  io.writeln(upper)
  <- 0
}
```

Sentinels: Nothing and Default

Zenlang has two explicit sentinel values:

Nothing (Absence of a Value)

Nothing represents the absence of any value:

```
let user -> Nothing
when user == Nothing {
  io.writeln(`No user logged in.`)
}
```

Default (The Type Zero-State)

Default represents the canonical zero-state of a type:

- ❖ 0 for numbers
- ❖ "" for strings
- ❖ [] for lists
- ❖ {} for maps
- ❖ False for booleans

```
let count -> Default // Initializes to 0
let items -> Default // Initializes to []
```

Both Nothing and Default are assignable and comparable with == and !=.

Lists

Lists are dynamic, ordered collections:

```
let fruits -> [`Apple`, `Banana`, `Cherry`]

// Accessing items (0-based)
let first -> fruits[0] // `Apple`

// Adding items
fruits.append(`Date`)

// Length
let total -> fruits.length // 4

// Iteration
let i -> 0
do while i < fruits.length {
  io.writeln(fruits[i])
  i -> 0 + i + 1
}
```

Maps

Maps are key-value associative dictionaries. Pairs are assigned with the -> flow arrow:

```

let user -> {
  `name` -> `Alice`,
  `role` -> `Admin`,
  `active` -> True
}

// Accessing fields
let username -> user[`name`]
let user_role -> user.role // Dot notation supported

// Setting fields
user[`last_login`] -> `2026-08-23`

// Checking existence safely
when user.keys().contains(`role`) {
  io.writeln(`User role is: ` + user.role)
}

```

❖ CHAPTER ❖

Control Flow & Looping



Zenlang provides clean, structured control flow statements: when for branching and do for looping.

Branching with when

Instead of if/elif/else, Zenlang uses when, or when, and or:

```

use zen.io
use zen.text.string as Str

function check_status(code) {
  when code == 200 {
    io.writeln(`OK: Success`)
  } or when code == 404 {
    io.writeln(`Not Found: Resource missing`)
  } or when code == 500 {
    io.writeln(`Error: Server failure`)
  } or {
    io.writeln(`Unknown status code: ` + Str.to_string(code))
  }
}

```

Single-Line when Guards

For compact checks and early returns:

```
function divide(a, b) {  
  when b == 0 { <- Nothing }  
  <- a / b  
}
```

Looping with `do while` and `do until`

Zenlang provides explicit loop constructs:

`do while` (**Runs while condition is True**)

```
let count -> 5  
do while count > 0 {  
  io.writeln(`Countdown: ` + Str.to_string(count))  
  count -> count - 1  
}  
io.writeln(`Blast off!`)
```

`do until` (**Runs until condition becomes True**)

```
let buffer -> []  
do until buffer.length >= 3 {  
  buffer.append(`item`)  
}
```

Loop Control: `break` and `continue`

- ❖ `break`: Exits the loop immediately.
- ❖ `continue`: Skips to the next iteration.

```
let i -> 0  
do while i < 10 {  
  i -> 0 + i + 1  
  when i == 5 { continue } // Skip 5  
  when i > 8 { break }      // Stop after 8  
  io.writeln(`Value: ` + Str.to_string(i))  
}
```

Pattern Matching & Destructuring

Zenlang supports expressive structural pattern matching:

Matching Lists & Spreads

```
let values -> [1, 2, 3]  
  
check values {  
  case [first, ..rest] {  
    io.writeln(`First: ` + Str.to_string(first))  
  }  
}
```

Matching Maps & Guards

```
let point -> { `x` -> 10, `y` -> 20 }

check point {
  case { `x` -> x, `y` -> y } when x > 0 {
    io.writeln(`Positive X coordinate: ` + Str.to_string(x))
  }
}
```

PART II

Abstractions & Architecture

Functions, snapshot closures, structures, classes, error handling, and modules.

❖ C H A P T E R ❖

Functions & Closures



Functions are the primary building blocks of Zenlang programs. They support first-class treatment, higher-order compositions, and safe snapshot-by-value closures.

Defining Functions

Functions are defined with the `function` keyword:

```
use zen.io
use zen.text.string as Str

function greet(name) {
  io.writeln(`Hello, ` + name + `!`)
}

function add(a, b) {
  <- a + b
}

function main() {
  greet(`Zen Programmer`)
  let sum -> add(10, 20)
  io.writeln(`Sum: ` + Str.to_string(sum))
  <- 0
}
```

First-Class & Anonymous Functions

In Zenlang, functions are first-class values: you can assign them to variables, pass them to other functions, and return them:

```
// Anonymous function / Lambda
let double -> function(x) {
  <- x * 2
}

let result -> double(21) // 42
```

Higher-Order Functions

A higher-order function is a function that accepts another function as an argument:

```
function apply_twice(fn, value) {
  let first -> fn(value)
  <- fn(first)
}

function increment(x) {
  <- x + 1
}

let val -> apply_twice(increment, 5) // 7
```

Closures & Snapshot-By-Value Capture

When an anonymous function references variables from its surrounding scope, it forms a **closure**.

In Zenlang, outer local variables are captured **by value (snapshot)**:

```
function make_adder(offset) {
  let adder -> function(x) {
    <- x + offset // offset is captured by value
  }
  <- adder
}

let add_10 -> make_adder(10)
let add_50 -> make_adder(50)

io.writeln(Str.to_string(add_10(5))) // Prints: 15
io.writeln(Str.to_string(add_50(5))) // Prints: 55
```

Why Snapshot-By-Value?

- ❖ **Concurrency Safety:** Tasks and closures cannot inadvertently mutate each other's outer stack variables.
- ❖ **Predictable Behavior:** The captured value remains constant even if the outer scope changes later.

❖ CHAPTER ❖

Structures, Classes & OOP



Zenlang supports both lightweight data structures and full object-oriented classes with methods and state encapsulation.

Structures (struct)

Structures are pure, named data records:

```
struct Point {  
  x: Integer,  
  y: Integer  
}  
  
function main() {  
  let p -> Point(10, 20)  
  io.writeln(`Point coordinates: ` + Str.to_string(p.x) + `, ` + Str.to_string(p.y))  
  <- 0  
}
```

Classes & Object-Oriented Programming (class)

Classes encapsulate both state and behavior. Methods receive an implicit `self` reference:

```
use zen.io  
use zen.text.string as Str  
  
class Counter {  
  let count -> 0  
  
  function increment() {  
    self.count -> self.count + 1  
    <- self.count  
  }  
  
  function decrement() {  
    self.count -> self.count - 1  
    <- self.count  
  }  
  
  function reset() {  
    self.count -> 0  
  }  
  
  function get_value() {  
    <- self.count  
  }  
}  
  
function main() {  
  let c -> Counter()  
  c.increment()  
  c.increment()  
  io.writeln(`Count is: ` + Str.to_string(c.get_value())) // 2  
  <- 0  
}
```

Constructors with Initialization

```
class Rectangle {
  let width -> 0
  let height -> 0

  function area() {
    <- self.width * self.height
  }

  function perimeter() {
    <- 2 * (self.width + self.height)
  }
}

function create_rect(w, h) {
  let r -> Rectangle()
  r.width -> w
  r.height -> h
  <- r
}
```

Algebraic Data Types (ADTs)

ADTs allow modeling domain entities as closed sets of variants:

```
enum Shape {
  Circle(radius: Decimal),
  Rect(width: Decimal, height: Decimal)
}

function describe_shape(shape) {
  check shape {
    case Shape.Circle(r) {
      io.writeln(`Circle with radius: ` + Str.to_string(r))
    },
    case Shape.Rect(w, h) {
      io.writeln(`Rectangle: ` + Str.to_string(w) + `x` + Str.to_string(h))
    }
  }
}
```

❖ C H A P T E R ❖

Error Handling & Recovery



In Zenlang, error handling is explicit, ergonomic, and structured. There are no silent crashes or untyped surprises.

Catching Errors with `check`

The `check` expression executes a block of code and catches any errors raised during execution:

```
use zen.io
use zen.text.string as Str

function safe_division(a, b) {
  let result -> check {
    when b == 0 { raise `DivisionByZero` }
    <- a / b
  } or {
    io.writeln(`Caught an error during division!`)
    <- 0
  }
  <- result
}

function main() {
  let res -> safe_division(10, 0)
  io.writeln(`Result: ` + Str.to_string(res)) // 0
  <- 0
}
```

Raising Errors with `raise`

Use `raise` to signal an exceptional condition:

```
function parse_port(s) {
  let port -> Str.to_integer(s)
  when port < 1 or port > 65535 {
    raise `InvalidPortNumber`
  }
  <- port
}
```

The Result Map Pattern

For operations that frequently encounter predictable errors (such as file I/O or network requests), the standard library idiom is the **Result Map**:

```
function read_user_file(filepath) {
  when not file.exists(filepath) {
    <- { `ok` -> False, `error` -> `File not found: ` + filepath }
  }
  let content -> file.read(filepath)
  <- { `ok` -> True, `data` -> content }
}

function main() {
  let res -> read_user_file(`config.json`)
  when res.ok {
    io.writeln(`File content: ` + res.data)
  } or {
    io.writeln(`Error: ` + res.error)
  }
  <- 0
}
```

❖ CHAPTER ❖

Modules, Packages & Reflection



As programs grow, structuring code into reusable modules and understanding runtime types through reflection becomes essential.

Modules & Packages (use)

Every `.zl` file is a module. To import and bring external packages into scope, use the canonical `use` keyword:

```
// Standard package imports (binds to the final segment: io, time, reflect)
use zen.io
use zen.time
use zen.reflect

// Qualified imports with explicit aliases
use zen.text.string as Str
use zen.math.math as Math
use zen.ui.table as table

// Relative module imports
use `./math_utils.zl` as utils
```

The Built-in module Object

Every Zenlang file has access to the implicit `module` object to query its execution environment:

```

use zen.io
use zen.text.string as Str

function main() {
  io.writeln(`Module Name: ` + module.name)
  io.writeln(`Module Path: ` + module.path)
  io.writeln(`Is Entry Point: ` + Str.to_string(module.is_entry))
  <- 0
}

```

Custom Module Entry Points

You can define custom start functions and set `module.entry`:

```

function run_server() {
  io.writeln(`Server daemon started!`)
  <- 0
}

module.entry -> `run_server`

```

Runtime Reflection with `zen.reflect`

The `zen.reflect` standard library package provides comprehensive introspection:

```

use zen.io
use zen.reflect
use zen.text.string as Str

function inspect_value(val) {
  let t -> reflect.type_name(val)
  io.writeln(`Type: ` + t)

  when reflect.is_list(val) {
    io.writeln(`Contains ` + Str.to_string(val.length) + ` elements`)
  } or when reflect.is_map(val) {
    io.writeln(`Keys: ` + Str.join(val.keys(), `, `))
  }
}

function main() {
  inspect_value(`Hello`)           // Type: String
  inspect_value([1, 2, 3])         // Type: List, Contains 3 elements
  inspect_value({ `a` -> 1, `b` -> 2 }) // Type: Map, Keys: a, b
  <- 0
}

```

PART III

Terminal Systems & Projects

High-speed ANSI canvas, dual execution under C, and full practical applications.

❖ C H A P T E R ❖

Terminal Mastery & UI



Zenlang treats the terminal as a primary graphical and interactive surface. In this chapter, you will learn how to build rich, zero-flicker terminal interfaces and CLI tools.

Terminal Control (zen.sys.term)

The `zen.sys.term` package gives you low-level, high-speed control over the terminal:

```
use zen.sys.term as term

function main() {
  term.alt_screen_enter() // Switch to clean alternate buffer
  term.raw_enter()        // Receive key events immediately
  term.clear()

  term.move(10, 5)        // Move cursor to (col 10, row 5)
  term.color(`green`, `black`)
  term.write(`Welcome to Zenlang TUI!`)

  let key -> term.read_key() // Wait for any key press

  term.raw_exit()
  term.alt_screen_exit()

  <- 0
}
```

Declarative Tables (zen.ui.table)

Formatting tabular data is effortless with `zen.ui.table`:

```
use zen.io
use zen.ui.table as table

function main() {
  let headers -> ['Process`, `CPU %`, `RAM (MB)`, `Status`]
  let rows -> [
    ['zen_compiler`, `12.5`, `45.2`, `Running`],
    ['postgres`, `4.1`, `210.0`, `Active`],
    ['web_server`, `0.8`, `32.4`, `Idle`]
  ]

  // Render with rounded Unicode borders
  let output -> table.format_table(headers, rows, { `style` -> `rounded` })
  io.writeln(output)
  <- 0
}
```

Animated Spinners (zen.ui.spinner)

To give users feedback during background tasks:

```
use zen.ui.spinner as spinner
use zen.sys.term as term

function main() {
  let s -> spinner.create(`Compiling project...`, `dots`)

  let step -> 0
  do while step < 20 {
    s.tick()
    term.poll_key(100) // Sleep 100ms
    step -> 0 + step + 1
  }

  s.success(`Build completed in 2.0s!`)
  <- 0
}
```

Visual Charts & Sparklines (zen.ui.chart)

Render real-time telemetry and metrics:

```

use zen.io
use zen.ui.chart as chart

function main() {
  let telemetry -> [10, 25, 45, 80, 95, 70, 40, 60, 85, 100]
  let spark -> chart.sparkline(telemetry)

  io.writeln(`Network Traffic: ` + spark)
  // Prints: Network Traffic: 
  <- 0
}

```

❖ CHAPTER ❖

Dual Execution & The C Runtime



Zenlang is built upon an architecture of **Dual Execution Parity**. Every Zenlang program can run in two complementary modes without changing a single line of code.

The Two Execution Modes

1. The Interpreter Path

❖ **Command:** `./bin/zen script.zl`

❖ **Use Case:** Rapid local development, debugging, interactive scripts, and REPL exploration.

2. The Native AOT Path (-g)

❖ **Command:** `./bin/zen -g script.zl`

❖ **Use Case:** Production deployments, high-performance CLI tools, standalone binary distribution.

Under the Hood: C Transpilation & SMIR

When compiling with `-g`, the compiler performs:

1. **Lexical & Parsing:** Builds the Abstract Syntax Tree (AST).
2. **Type Checking & Inference:** Validates variable scopes and type annotations.
3. **SMIR Analyzer:** Converts AST into Structured Machine Intermediate Representation (SMIR), tracking variable lifetimes and move semantics.
4. **C Codegen:** Emits clean, optimized C code linking to the shared C runtime in `runtime/`.
5. **Native Linking:** Invokes `gcc` or `clang` with `-O3` optimizations.

The Shared C Runtime (runtime/)

The C runtime provides:

- ❖ **ZenValue:** A tagged union representation of all Zenlang types (Integers, Decimals, Strings, Lists, Maps, Functions, Objects).
- ❖ **Automated Memory Safety:** Structured reference tracking without pause-the-world garbage collection overhead.

- ❖ **Platform Abstraction:** POSIX and Windows terminal abstractions for raw keyboard polling and alt-screen switching.

The Road to Self-Hosting (`selfhost/`)

The ultimate milestone for Zenlang is full self-hosting:

- ❖ Phase 1: Porting Tokenizer & Lexer to Zenlang (`selfhost/compiler/lexer.zl`)
- ❖ Phase 2: Porting Parser & AST nodes (`selfhost/compiler/parser.zl`)
- ❖ Phase 3: Self-compilation where `selfhost/zen.zl` compiles itself!

❖ C H A P T E R ❖

Practical Projects



In this final chapter, we will synthesize everything we have learned into two complete, practical applications.

Project 1: Unix Log Analyzer (`zlog.zl`)

A high-speed CLI tool that parses server log files, counts HTTP status codes, and outputs a formatted table with a visual frequency distribution:

```

// zlog.zl
use zen.io
use zen.io.file as file
use zen.sys.sys as sys
use zen.text.string as Str
use zen.ui.table as table

function main() {
  let args -> sys.get_args()
  when args.length < 2 {
    io.writeln(`Usage: zlog <logfile.log>`)
    <- 1
  }

  let filepath -> args[1]
  when not file.exists(filepath) {
    io.writeln(`File not found: ` + filepath)
    <- 1
  }

  let lines -> file.read_lines(filepath)
  let counts -> { `200` -> 0, `404` -> 0, `500` -> 0, `other` -> 0 }

  let i -> 0
  do while i < lines.length {
    let line -> lines[i]
    when Str.contains(line, ` 200 `) {
      counts[`200`] -> counts[`200`] + 1
    } or when Str.contains(line, ` 404 `) {
      counts[`404`] -> counts[`404`] + 1
    } or when Str.contains(line, ` 500 `) {
      counts[`500`] -> counts[`500`] + 1
    } or {
      counts[`other`] -> counts[`other`] + 1
    }
    i -> 0 + i + 1
  }

  let headers -> [ `Status Code`, `Count`, `Description` ]
  let rows -> [
    [ `200 OK`, Str.to_string(counts[`200`]), `Successful HTTP requests` ],
    [ `404 Not Found`, Str.to_string(counts[`404`]), `Missing resource requests` ],
    [ `500 Server Error`, Str.to_string(counts[`500`]), `Internal server errors` ],
    [ `Other`, Str.to_string(counts[`other`]), `Redirects / Other` ]
  ]

  io.writeln(`\n--- ACCESS LOG ANALYSIS: ` + filepath + ` ---`)
  io.writeln(table.format_table(headers, rows, { `style` -> `rounded` }))
  <- 0
}

```

Project 2: Interactive Terminal Task Manager (ztodo.zl)

A split-screen interactive task list with live keyboard navigation:

```

// ztodo.zl
use zen.io
use zen.sys.term as term
use zen.text.string as Str

class TodoApp {
  let tasks -> [
    { `title` -> `Write Zenlang book chapters`, `done` -> True },
    { `title` -> `Test dual-path native compilation`, `done` -> True },
    { `title` -> `Ship 1.0 release`, `done` -> False }
  ]
  let selected -> 0
  let running -> True

  function draw() {
    term.move(1, 1)
    term.write(`\033[30;102m 📝 ZEN TODO MANAGER - [Space:Toggle ↑/↓:Nav Q:Quit]
\033[0m\n\n`)

    let i -> 0
    do while i < self.tasks.length {
      let task -> self.tasks[i]
      let prefix -> ` `
      when i == self.selected { prefix -> `▶ ` }

      let checkmark -> `[ ] `
      when task.done { checkmark -> `[✓] ` }

      when i == self.selected {
        term.write(`\033[36m` + prefix + checkmark + task.title + `\033[0m\n`)
      } or {
        term.write(prefix + checkmark + task.title + `\n`)
      }
      i -> 0 + i + 1
    }
  }

  function run() {
    term.alt_screen_enter()
    term.raw_enter()
    term.clear()

    do while self.running {
      self.draw()
      let key -> term.read_key()
      when key.name == `up` or key.ch == `k` {
        when self.selected > 0 { self.selected -> self.selected - 1 }
      } or when key.name == `down` or key.ch == `j` {
        when self.selected < self.tasks.length - 1 { self.selected ->
self.selected + 1 }
      } or when key.ch == ` ` {
        self.tasks[self.selected].done -> not self.tasks[self.selected].done
      } or when key.ch == `q` or key.ch == `Q` {
        self.running -> False
      }
    }
  }
}

```